

LEVEL 4 — DEVELOPMENT TRACK FOUNDATIONS

Git & Version Control

Tracking, reviewing and collaborating on changes to code and research safely

Covers: Repository structure · branching · commits · pull requests · code review · tags/releases · GitHub/GitLab workflows

Table of Contents

Right-click and choose “Update Field” (or “Update entire table”) after opening in Word to populate page numbers.

Table of Contents	2
10. Git & Version Control (Level 4)	3
10.1 Learning Position at a Glance	3
10.2 Repository Structure	3
10.3 Branching	4
10.4 Commits	5
10.5 Pull Requests & Code Review	5
10.6 Tags & Releases	6
10.7 GitHub/GitLab Workflows	7
10.8 Worked Exercise — Model Answer	7
10.9 Common Mistakes — Deeper Look	8
10.10 Advanced Extensions	8
10.11 Professional Applications	9

10. Git & Version Control (Level 4)

Covers: Repository structure · branching · commits · pull requests · code review · tags/releases · GitHub/GitLab workflows

10.1 Learning Position at a Glance

- **What it is:** The discipline and tooling for tracking, reviewing and collaborating on changes to code and research.
- **Why it matters:** Without disciplined version control, research results become irreproducible and collaborative development becomes unsafe.
- **Prerequisites:** Software Engineering Foundations.
- **Connects to Research:** Tagging the exact code version used to produce a given backtest result is essential for reproducibility.
- **Connects to Development:** Branching strategy and code review are the backbone of safe, collaborative development.
- **Connects to Trading:** Release tagging matters when diagnosing whether a live trading issue was caused by a recent code change.
- **Practical exercise:** Set up a repository with a clear branching strategy (e.g. main/develop/feature branches), and practise a full pull-request-and-review cycle with a small code change.
- **Common beginner mistakes:**
 - Committing large binary data files directly into Git instead of a proper data-versioning tool.
 - Working directly on the main branch with no review process.
- **Advanced extensions:** Monorepo vs polyrepo strategy for a growing quant research organisation.
- **Professional applications:** Reproducible research; safe collaborative development.

10.2 Repository Structure

A repository is the full, tracked history of a project: every version of every file that has ever been committed, plus the metadata (who, when, why) describing how they changed. How a repository's files and folders are organised has a direct effect on how easy the project is to navigate, test and deploy.

Example 10.1 — A repository structure that separates production code from exploration

```
quant-research/  
├── src/  
│   ├── signals/           # Signal implementations (§9.4)  
│   ├── portfolio/        # Portfolio and risk logic  
│   └── execution/         # Backtester and execution simulation  
├── tests/                 # Unit and integration tests, mirroring src/  
├── notebooks/             # Exploratory research (never imported by src/)  
├── configs/               # Externalised configuration (§9.8)  
├── .gitignore  
├── README.md  
└── pyproject.toml
```

Two structural habits matter most in a research context: keeping exploratory notebooks physically separate from importable, tested source code (so a notebook experiment can never silently become a hidden production dependency), and a .gitignore file that excludes anything that should not be tracked at all — generated output, credentials, and, critically, large data files (see the common mistake in §10.9).

Example 10.2 — A .gitignore that keeps large data and secrets out of the repository

```
# .gitignore  
__pycache__/  
*.pyc  
.env
```

```
*.csv
*.parquet
/data/raw/
/notebooks/.ipynb_checkpoints/
```

10.3 Branching

A branch is an independent line of development — a way to work on a change without affecting the stable, shared codebase until that change is ready. A simple and widely effective strategy uses three kinds of branch: a main branch that always reflects what is deployed (or ready to deploy), a develop branch that integrates finished features ahead of a release, and short-lived feature branches, one per piece of work, that are merged in and then deleted.

A simple main / develop / feature branching strategy

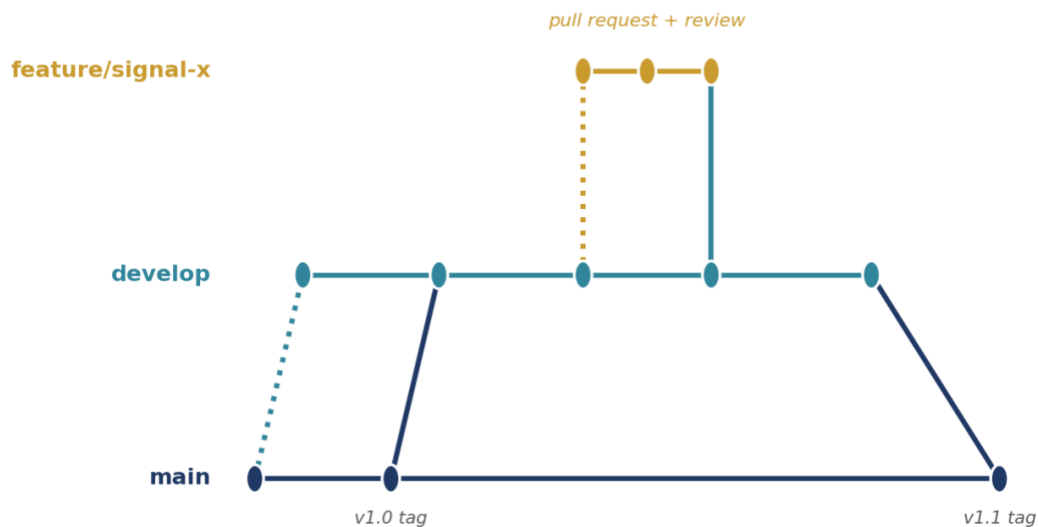


Figure 10.1 — A feature branch is created from develop, worked on independently, and merged back through a reviewed pull request; develop is periodically merged into main and tagged as a release.

Example 10.3 — Creating and pushing a feature branch

```
git checkout develop
git pull
git checkout -b feature/reversal-signal

# ... make changes, then commit them (see §10.4) ...

git push -u origin feature/reversal-signal
# then open a pull request from feature/reversal-signal into develop
```

- **main:** Always deployable; changes only arrive here through a reviewed merge from develop (or a hotfix branch, for urgent live fixes).
- **develop:** Integrates completed, reviewed features ahead of the next release; the default base for new feature branches.
- **feature/*:** One branch per piece of work, named descriptively, deleted once merged — kept short-lived to minimise how far it can drift from develop before merging back.

10.4 Commits

A commit is a saved, named snapshot of the repository at a point in time. Good commit hygiene — small, logically self-contained commits with a clear message — makes the project's history genuinely useful: a bug can be traced to the specific commit that introduced it (via `git bisect`), and a reviewer can follow the reasoning behind a change rather than facing one enormous, undifferentiated diff.

Example 10.4 — A well-scoped commit vs. a vague, oversized one

```
# A well-scoped, clearly described commit
git add src/signals/reversal.py tests/test_reversal.py
git commit -m "Add 5-day reversal signal with unit tests"

# Bad practice: one giant commit mixing unrelated changes
git add .
git commit -m "updates"    # what updates? to what? why?
```

- **Atomic:** Each commit represents one logical change; a reviewer (or a future `git bisect` run) can understand and, if needed, revert it in isolation.
- **Descriptive:** The commit message states what changed and, where it is not obvious, why — ‘fix bug’ tells a future reader nothing that the diff does not already show.
- **Buildable:** Each commit should leave the codebase in a working state — tests passing — so that bisecting or reverting to any historical commit is safe.

10.5 Pull Requests & Code Review

A pull request (PR, or merge request on GitLab) proposes merging one branch into another and provides a dedicated place to review the change before it happens: a diff of every change, a space for comments, and a record of who approved it and when. Code review — a second person reading and commenting on the change before it merges — is the single most effective, low-cost safeguard against the kind of silent bug described in §9.6, because it puts a second pair of eyes on the change before it reaches shared code.

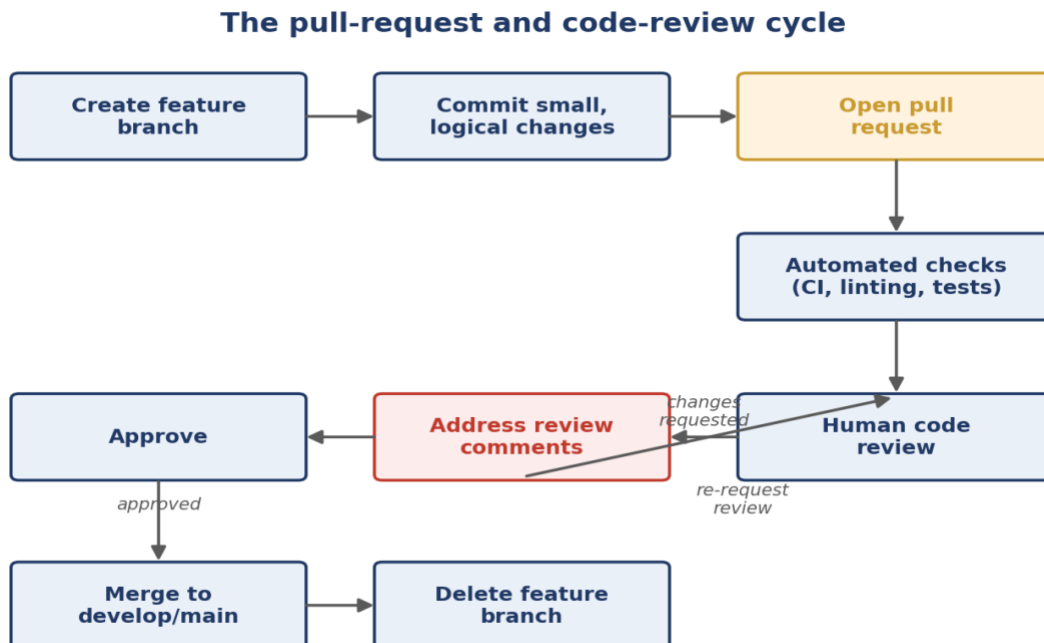


Figure 10.2 — The full pull-request cycle: automated checks catch mechanical issues (failing tests, style violations) immediately, while human review catches the things automation cannot — whether the change is correct, well-designed, and does what it claims to.

What a good review comments on

- Correctness: does the change actually do what the description says, including edge cases?
- Design: does it fit the SOLID and pattern discipline from §9.2–§9.4, or does it introduce unnecessary coupling or duplication?
- Tests: are the new tests meaningful, or do they merely execute the code without checking anything?
- Clarity: could the next person (including the author, in six months) understand this without asking?

Example 10.5 — A pull-request description that gives a reviewer everything needed to review it well

```
## Pull request description

**What**: Adds a 5-day mean-reversion signal alongside the existing
20-day momentum signal.

**Why**: Backtests (see linked research note) show a modest,
economically-motivated diversification benefit when combined with
the existing momentum signal (see §6.2 for why the economic
rationale is stated explicitly).

**How tested**: Added unit tests for the new signal class; ran the
full backtest suite locally, no regressions.
```

10.6 Tags & Releases

A tag is a fixed, named pointer to one specific commit — unlike a branch, it does not move as new commits are added. Tags are how a specific, exact version of the codebase is given a memorable, permanent name, most commonly to mark a release.

Example 10.6 — Creating and checking out an annotated tag

```
git tag -a v1.2.0 -m "Add risk-limit change ahead of Q3 deployment"
git push origin v1.2.0

# Later, to see exactly what code was live at that tag:
git checkout v1.2.0
```

Tags mark exactly which commit was live at a given time



Figure 10.3 — Tags give every deployed version a permanent, memorable name; when a live issue appears, the commits between the last known-good tag and the current tag are exactly the candidate causes.

- **For research reproducibility:** Recording the exact tag (or commit hash) used to produce a given backtest result means that result can be regenerated exactly, even if the codebase has moved on substantially since — directly supporting the reproducibility discipline in §6.5.
- **For diagnosing live issues:** When a trading issue appears, comparing the current live tag against the previous one immediately narrows the search to a specific, bounded set of commits, rather than the entire history of the codebase.

10.7 GitHub/GitLab Workflows

GitHub and GitLab layer collaboration tooling on top of Git itself: hosted repositories, pull/merge requests, code review tools, and automation (continuous integration, or CI) that runs tests and checks automatically on every proposed change. A typical CI configuration runs the test suite, a type checker (§9.5) and a linter automatically on every pull request, so a reviewer's time is spent on design and correctness rather than mechanical issues a machine can catch faster and more reliably.

Example 10.7 — A minimal CI configuration that gates every pull request on type checks and tests

```
# .github/workflows/ci.yml (simplified)
name: CI
on: [pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: pip install -e .[dev]
      - run: mypy src/
      - run: pytest tests/
```

- **Branch protection rules:** Configuration that prevents pushing directly to main or develop and requires passing CI checks and at least one approval before a pull request can be merged — the direct technical enforcement of the code-review discipline in §10.5.
- **Issue tracking:** Linking commits and pull requests to a tracked issue ('fixes #142') keeps a record of why a change was made, connected directly to the code that made it.

10.8 Worked Exercise — Model Answer

Exercise: Set up a repository with a clear branching strategy (e.g. main/develop/feature branches), and practise a full pull-request-and-review cycle with a small code change.

Model answer structure

Model answer

```
# 1. Initialise the repository with main and develop
git init
git add .
git commit -m "Initial commit"
git branch develop

# 2. Create a feature branch from develop for a small, real change
git checkout develop
git checkout -b feature/add-sharpe-docstring

# 3. Make a small, well-scoped change and commit it
# (e.g. adding the docstring from Example 9.9)
git add src/metrics.py
git commit -m "Document edge-case behaviour of sharpe_ratio"
git push -u origin feature/add-sharpe-docstring
```

```
# 4. Open a pull request into develop, using the PR template
#    from Example 10.5, and request review from a colleague

# 5. Address any review comments with new commits on the same branch,
#    then merge once approved and CI passes, and delete the branch
git branch -d feature/add-sharpe-docstring
```

The exercise is complete once all five steps in Figure 10.2 have genuinely been exercised at least once: a branch was created, a small logical commit was made, a real pull request was opened with a description a reviewer could act on, at least one round of review feedback was given and addressed, and the branch was merged and cleaned up — not merely read about.

10.9 Common Mistakes — Deeper Look

Mistake: Committing large binary data files directly into Git instead of a proper data-versioning tool

Git is designed to track line-by-line changes to text files efficiently; a large binary file (a CSV of tick data, a trained model checkpoint) has no meaningful ‘diff’ for Git to compute, so every version of that file is stored in full, and the repository’s size grows without bound as the file changes over time. Beyond the practical slowdown (cloning, fetching and checking out the repository all get progressively slower), this conflates two genuinely different problems — versioning code and versioning data — that call for different tools: Git for code, and a dedicated data-versioning tool (or the point-in-time database vintages discussed in §6.4) for large or frequently-changing datasets.

Example 10.8 — What not to commit, and why

```
# Do not do this:
git add data/tick_data_2024.csv    # hundreds of MB, grows every version
git commit -m "Add tick data"

# Instead: keep data out of Git entirely (see the .gitignore in
# Example 10.2) and reference it by a versioned path or dataset ID
# managed by a dedicated data-versioning tool.
```

Mistake: Working directly on the main branch with no review process

Committing directly to main skips every safeguard this section has described: no pull request means no diff for a reviewer to read, no automated CI check gating the change, and no record of who approved it or why. A single unreviewed change to position-sizing or risk logic pushed straight to main is exactly the kind of silent, unvetted change that §9.1’s opening warning — that a single bug in this code can lose real money — is about.

⚠ Case study: a direct-to-main commit that reached production

- A researcher, under time pressure, pushed a one-line change to a position-sizing function directly to main to ‘save time’ rather than opening a pull request, bypassing both CI and human review.
- The change contained a sign error that inverted the direction of a risk adjustment; because no reviewer or automated check ever saw the diff, the error reached the live trading system unnoticed.
- The firm’s response was not to add more manual vigilance but to enable branch protection rules (§10.7) making it technically impossible to push directly to main, ensuring every future change — however small, however urgent — passes through review and CI by construction.

10.10 Advanced Extensions

Monorepo vs. polyrepo strategy for a growing quant research organisation: a monorepo keeps all projects (research, execution, risk, infrastructure) in a single repository, while a polyrepo splits them into many independent repositories. Neither is universally correct; the trade-off shifts as an organisation grows.

Dimension	Monorepo	Polyrepo
Cross-project changes	A single pull request can change a shared library and every consumer of it atomically	Requires coordinating separate pull requests across multiple repositories
Code sharing	Shared utilities are trivially importable across all projects	Shared code must be published as a versioned package and explicitly updated by each consumer
Access control	Harder to restrict who can see or change specific projects	Naturally supports fine-grained, per-repository access control
Tooling at scale	Requires investment in tooling (selective builds/tests) to remain fast as the repository grows very large	Each repository stays small and fast by construction, at the cost of duplicated tooling setup

10.11 Professional Applications

- **Reproducible research:** Every reported backtest result should be traceable to an exact commit or tag (§6.5, §10.6), so it can be regenerated exactly and audited later, even long after the codebase has continued to evolve.
- **Safe collaborative development:** Branch protection, mandatory review, and CI (§10.5, §10.7) together make it structurally difficult for an unreviewed, untested change to reach a shared codebase or a live trading system — turning the discipline described in this section from a best-practice suggestion into an enforced default.